ASSIGNMENT-16.3

HT.NO-2403A52L04

BATCH-50

QUESTION:

Lab 16 – Database Design and Queries: Schema Design and SQL

Generation

Lab Objectives:

• Understand schema design principles for relational databases.

Week8 -

Wednesday

• Use AI-assisted tools to generate SQL schema definitions.

• Write and refine SQL queries (CRUD + aggregate operations).

• Explore how AI can optimize queries and ensure normalization.

Task 1 – Hospital Management Database

Task:

Create schema and queries for a Hospital Management System.

Instructions:

• Tables: Doctors, Patients, Appointments.

• Use AI to define constraints (unique IDs, valid dates).

• Generate queries:

• List all appointments for a specific doctor.

• Retrieve patient history by patient ID.

• Count total patients treated by each doctor.

Expected Output:

• Normalized schema and SQL queries with joins.

Task 2 – Real-Time Application: Online Shopping Database

Scenario:

Design a database for an E-commerce platform.

Requirements:

• Tables: Users, Products, Orders, OrderDetails.

• Generate queries to:

• Retrieve all orders by a user.

• Find the most purchased product.

• Calculate total revenue in a given month.

• AI should also suggest normalization improvements and query optimization.

Expected Output:

• Complete schema with relationships + SQL queries for analytics.

Task 3 – Library Database

Task:

Design schema for a Library Management System.

Instructions:

• Tables: Books, Members, Loans.

• Use AI to suggest indexing strategy for faster queries.

• Generate queries:

• Retrieve all books currently issued.

• Find overdue books (loan date > 30 days).

• Count number of books loaned by each member.

Expected Output:

• Schema + SQL queries demonstrating joins and conditions.

Task 4: Optimize Queries

Instructions:

• Use AI tools to analyze query efficiency.

• Optimize inefficient queries using joins, indexes, or reducing subqueries.

• Test optimized queries for correctness.

Starter Code Example:

-- Original query

SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM Authors WHERE country='UK');

-- Optimized query

SELECT b.*

FROM Books b

JOIN Authors a ON b.author_id = a.author_id

WHERE a.country = 'UK';

Expected Output:

• Both queries return the same result: all books written by UK authors.

• Optimized query performs faster for larger datasets.

Task 5: University Course Registration

Scenario:

SR University needs a course registration system for students enrolling in different courses under faculty supervision.

• Use Copilot to design schema tables: Students, Courses, Faculty, and Registrations.

• Define relationships:

• Each student can register for multiple courses.

• Each course is taught by a faculty member.

• Write SQL queries to:

• List all students enrolled in a specific course.

• Find faculty members teaching more than 2 courses.

• Retrieve courses with the highest number of registrations.

CODE:

```
import sqlite3
from datetime import datetime, timedelta


# Create database
conn = sqlite3.connect("lab16.db")
```

```
cursor = conn.cursor()

print("\n✅ DATABASE CREATED\n")

# ---------------- TASK 1: HOSPITAL ----------------
print("===== TASK 1: HOSPITAL MANAGEMENT =====")

cursor.executescript("""
DROP TABLE IF EXISTS Doctors;
DROP TABLE IF EXISTS Patients;
DROP TABLE IF EXISTS Appointments;

CREATE TABLE Doctors(
    doctor_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    specialization TEXT
);

CREATE TABLE Patients(
    patient_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    age INTEGER
);

CREATE TABLE Appointments(
    appointment_id INTEGER PRIMARY KEY,
    doctor_id INTEGER,
    patient_id INTEGER,
    date TEXT,
```

```
    FOREIGN KEY(doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY(patient_id) REFERENCES Patients(patient_id)
);
""")

# Insert sample data
cursor.executemany("INSERT INTO Doctors VALUES (?,?,?)", [
    (1,"Dr. A","Cardio"),
    (2,"Dr. B","Neuro")
])

cursor.executemany("INSERT INTO Patients VALUES (?,?,?)", [
    (1,"John",30),
    (2,"Alice",25)
])

cursor.executemany("INSERT INTO Appointments VALUES (?,?,?,?)", [
    (1,1,1,"2026-01-01"),
    (2,1,2,"2026-02-01"),
    (3,2,1,"2026-03-01")
])

# Queries
print("\nAppointments for Doctor 1:")
for row in cursor.execute("""
SELECT * FROM Appointments WHERE doctor_id=1
"""):
    print(row)
```

```
print("\nPatient History (ID=1):")
for row in cursor.execute("""
SELECT * FROM Appointments WHERE patient_id=1
"""):
    print(row)

print("\nPatients treated per doctor:")
for row in cursor.execute("""
SELECT doctor_id, COUNT(DISTINCT patient_id)
FROM Appointments
GROUP BY doctor_id
"""):
    print(row)

# ---------------- TASK 2: E-COMMERCE ----------------
print("\n===== TASK 2: E-COMMERCE =====")

cursor.executescript("""
DROP TABLE IF EXISTS Users;
DROP TABLE IF EXISTS Products;
DROP TABLE IF EXISTS Orders;
DROP TABLE IF EXISTS OrderDetails;

CREATE TABLE Users(
    user_id INTEGER PRIMARY KEY,
    name TEXT
);

CREATE TABLE Products(
```

```
    product_id INTEGER PRIMARY KEY,
    name TEXT,
    price REAL
);

CREATE TABLE Orders(
    order_id INTEGER PRIMARY KEY,
    user_id INTEGER,
    date TEXT,
    FOREIGN KEY(user_id) REFERENCES Users(user_id)
);

CREATE TABLE OrderDetails(
    order_detail_id INTEGER PRIMARY KEY,
    order_id INTEGER,
    product_id INTEGER,
    quantity INTEGER,
    FOREIGN KEY(order_id) REFERENCES Orders(order_id),
    FOREIGN KEY(product_id) REFERENCES Products(product_id)
);
""")

# Insert data
cursor.executemany("INSERT INTO Users VALUES (?,?)", [
    (1,"User1"), (2,"User2")
])

cursor.executemany("INSERT INTO Products VALUES (?,?,?)", [
    (1,"Laptop",50000),
```

```
    (2,"Phone",20000)
])

cursor.executemany("INSERT INTO Orders VALUES (?,?,?)", [
    (1,1,"2026-04-01"),
    (2,1,"2026-04-02")
])

cursor.executemany("INSERT INTO OrderDetails VALUES (?,?,?,?)", [
    (1,1,1,1),
    (2,2,2,2)
])

# Queries
print("\nOrders by User 1:")
for row in cursor.execute("""
SELECT * FROM Orders WHERE user_id=1
"""):
    print(row)

print("\nMost purchased product:")
for row in cursor.execute("""
SELECT product_id, SUM(quantity) as total
FROM OrderDetails
GROUP BY product_id
ORDER BY total DESC LIMIT 1
"""):
    print(row)
```

```
print("\nTotal revenue:")
for row in cursor.execute("""
SELECT SUM(p.price * od.quantity)
FROM OrderDetails od
JOIN Products p ON od.product_id=p.product_id
"""):
    print(row)

# ---------------- TASK 3: LIBRARY ----------------
print("\n===== TASK 3: LIBRARY =====")

cursor.executescript("""
DROP TABLE IF EXISTS Books;
DROP TABLE IF EXISTS Members;
DROP TABLE IF EXISTS Loans;

CREATE TABLE Books(
    book_id INTEGER PRIMARY KEY,
    title TEXT
);

CREATE TABLE Members(
    member_id INTEGER PRIMARY KEY,
    name TEXT
);

CREATE TABLE Loans(
    loan_id INTEGER PRIMARY KEY,
    book_id INTEGER,
```

```
    member_id INTEGER,
    loan_date TEXT,
    FOREIGN KEY(book_id) REFERENCES Books(book_id),
    FOREIGN KEY(member_id) REFERENCES Members(member_id)
);
""")

# Insert data
today = datetime.now()
old_date = today - timedelta(days=40)

cursor.executemany("INSERT INTO Books VALUES (?,?)", [
    (1,"Book A"), (2,"Book B")
])

cursor.executemany("INSERT INTO Members VALUES (?,?)", [
    (1,"Member1"), (2,"Member2")
])

cursor.executemany("INSERT INTO Loans VALUES (?,?,?,?)", [
    (1,1,1,str(today.date())),
    (2,2,2,str(old_date.date()))
])

# Queries
print("\nBooks issued:")
for row in cursor.execute("SELECT * FROM Loans"):
    print(row)
```

```
print("\nOverdue books (>30 days):")
for row in cursor.execute("""
SELECT * FROM Loans
WHERE julianday('now') - julianday(loan_date) > 30
"""):
    print(row)

print("\nBooks per member:")
for row in cursor.execute("""
SELECT member_id, COUNT(*)
FROM Loans
GROUP BY member_id
"""):
    print(row)

# ---------------- TASK 4: QUERY OPTIMIZATION ----------------
print("\n===== TASK 4: OPTIMIZATION =====")

cursor.executescript("""
DROP TABLE IF EXISTS Authors;

CREATE TABLE Authors(
    author_id INTEGER PRIMARY KEY,
    name TEXT,
    country TEXT
);

INSERT INTO Authors VALUES (1,'Author1','UK');
INSERT INTO Authors VALUES (2,'Author2','US');
```

```
ALTER TABLE Books ADD COLUMN author_id INTEGER;
UPDATE Books SET author_id=1 WHERE book_id=1;
UPDATE Books SET author_id=2 WHERE book_id=2;
""")

print("\nOriginal Query:")
for row in cursor.execute("""
SELECT * FROM Books WHERE author_id IN
(SELECT author_id FROM Authors WHERE country='UK')
"""):
    print(row)

print("\nOptimized Query:")
for row in cursor.execute("""
SELECT b.*
FROM Books b
JOIN Authors a ON b.author_id = a.author_id
WHERE a.country='UK'
"""):
    print(row)

# ---------------- TASK 5: UNIVERSITY ----------------
print("\n===== TASK 5: UNIVERSITY =====")

cursor.executescript("""
DROP TABLE IF EXISTS Students;
DROP TABLE IF EXISTS Courses;
DROP TABLE IF EXISTS Faculty;
```

```
DROP TABLE IF EXISTS Registrations;

CREATE TABLE Students(
    student_id INTEGER PRIMARY KEY,
    name TEXT
);

CREATE TABLE Faculty(
    faculty_id INTEGER PRIMARY KEY,
    name TEXT
);

CREATE TABLE Courses(
    course_id INTEGER PRIMARY KEY,
    name TEXT,
    faculty_id INTEGER,
    FOREIGN KEY(faculty_id) REFERENCES Faculty(faculty_id)
);

CREATE TABLE Registrations(
    reg_id INTEGER PRIMARY KEY,
    student_id INTEGER,
    course_id INTEGER,
    FOREIGN KEY(student_id) REFERENCES Students(student_id),
    FOREIGN KEY(course_id) REFERENCES Courses(course_id)
);
""")

# Insert data
```

```
cursor.executemany("INSERT INTO Students VALUES (?,?)", [
    (1,"Student1"), (2,"Student2")
])

cursor.executemany("INSERT INTO Faculty VALUES (?,?)", [
    (1,"Prof A"), (2,"Prof B")
])

cursor.executemany("INSERT INTO Courses VALUES (?,?,?)", [
    (1,"Math",1),
    (2,"CS",1),
    (3,"Physics",2)
])

cursor.executemany("INSERT INTO Registrations VALUES (?,?,?)", [
    (1,1,1),
    (2,2,1),
    (3,1,2)
])

# Queries
print("\nStudents in Course 1:")
for row in cursor.execute("""
SELECT s.name
FROM Students s
JOIN Registrations r ON s.student_id=r.student_id
WHERE r.course_id=1
"""):
    print(row)
```

```
print("\nFaculty teaching >2 courses:")
for row in cursor.execute("""
SELECT faculty_id, COUNT(*)
FROM Courses
GROUP BY faculty_id
HAVING COUNT(*) > 2
"""):
    print(row)

print("\nMost popular course:")
for row in cursor.execute("""
SELECT course_id, COUNT(*) as total
FROM Registrations
GROUP BY course_id
ORDER BY total DESC LIMIT 1
"""):
    print(row)

conn.commit()
conn.close()

print("\n✅ ALL TASKS COMPLETED SUCCESSFULLY")
```

OUTPUT:

```
PS C:\Users\akhir\OneDrive\Documents\AI ASSISTANT> & C:\Users\akhir\AppData\Local\Python\pythoncore-3.14-64\python.exe

✅ DATABASE CREATED

===== TASK 1: HOSPITAL MANAGEMENT =====

Appointments for Doctor 1:
(1, 1, 1, '2026-01-01')
(2, 1, 2, '2026-02-01')

Patient History (ID=1):
(1, 1, 1, '2026-01-01')
(3, 2, 1, '2026-03-01')

Patients treated per doctor:
(1, 2)
(2, 1)

===== TASK 2: E-COMMERCE =====

Orders by User 1:
(1, 1, '2026-04-01')
(2, 1, '2026-04-02')

Most purchased product:
(2, 2)

Total revenue:
(90000.0,)
```

```
===== TASK 3: LIBRARY =====

Books issued:
(1, 1, 1, '2026-04-14')
(2, 2, 2, '2026-03-05')

Overdue books (>30 days):
(2, 2, 2, '2026-03-05')

Books per member:
(1, 1)
(2, 1)

===== TASK 4: OPTIMIZATION =====

Original Query:
(1, 'Book A', 1)

Optimized Query:
(1, 'Book A', 1)

===== TASK 5: UNIVERSITY =====

Students in Course 1:
('Student1',)
('Student2',)

Faculty teaching >2 courses:

Most popular course:
(1, 2)

✅ ALL TASKS COMPLETED SUCCESSFULLY
PS C:\Users\akhir\OneDrive\Documents\AI ASSISTANT>
```